Ideation Phase
Brainstorming & Idea Prioritization

Date	01 October 2026
Team ID	SWTID-2026-8643
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value (ServiceNow)
Maximum Marks	4 Marks

Step-1: Team Gathering, Collaboration and Problem Statement

Team gathering: all team members (2–8 people) participate in the session, review the ServiceNow lab brief in advance, and follow brainstorming rules such as staying focused, avoiding early judgment, encouraging creative ideas, generating many ideas, listening to others and using visuals where useful.

Goal: determine how to protect records in a custom ServiceNow table so that access is controlled by both the user's role and a field value in each record.

Problem statement (How Might We):

How might we limit ServiceNow users from reading, creating, editing and deleting records in the Institution Details table (u_institution_details) unless they have the required role and the record's Branch field satisfies a condition (for example, Branch = EEE), while administrators continue to have complete access?

Step-2: Brainstorm, Idea Listing and Grouping

Brainstorm: each member contributed ideas aimed at solving the stated problem.

Member	Ideas
Person 1	Create a test user (EEE User) and custom roles bb1–bb4 Create a custom table using Branch as a choice field (ECE, EEE, CSE)
Person 2	Read ACL with Advanced = true and a script using gs.hasRole() Apply a data condition where Branch is EEE so that only EEE records are considered
Person 3	Separate ACLs per operation: create (bb2), write (bb3), delete (bb4) Use role stacking so each additional privilege is built on the bb1 role
Person 4	Elevate security_admin to create ACLs Verify the setup by impersonating the EEE user, a user without roles, and an administrator Retain the administrator override so admins can always access every record
Person 5	Alternative: Data-condition-only ACL without script Alternative: Consider domain separation or a query business rule for filtering lists
Person 6	Alternative: field-level ACLs to hide sensitive columns (email, phone) Consider extending the solution later to Service Portal widgets and forms

Group ideas: related ideas were combined into clusters, with each cluster given a clear descriptive label.

Cluster	Ideas grouped
Set up users, roles and data	EEE User, roles bb1–bb4, u_institution_details table, sample records for ECE / EEE / CSE
Control who can see records (READ)	Record ACL with bb1 role, data condition Branch is EEE, advanced script, admin override
Control who can change records (CREATE / WRITE / DELETE)	Separate ACLs using bb2, bb3 and bb4 with no data condition
Test and validate	Impersonate EEE user, role-less user and admin; check New button, inline edit and delete
Alternatives and future scope	Data-condition-only ACL, query business rule, field-level ACLs, Service Portal coverage

Step-3: Idea Prioritization

Ideas were evaluated using an Importance vs Feasibility grid (1 = low, 5 = high). Importance measures the expected positive impact, while feasibility measures how easy an idea is to implement considering cost, time, effort and complexity.

Idea	Importance	Feasibility	Decision
Script-controlled READ ACL (role + Branch = EEE + admin override)	5	4	Do first (high importance, high feasibility)
WRITE ACL with bb3	4	5	Do first
CREATE ACL with bb2	4	5	Do first
DELETE ACL with bb4	4	5	Do first
Test via impersonation (EEE user, no role, admin)	5	5	Do first
Field-level ACLs for email / phone	3	4	Do later
Query business rule for list filtering	3	3	Do later
Service Portal coverage	3	2	Future scope
Domain separation	2	1	Not selected

Final selected solution:

- Create the EEE User account and roles bb1, bb2, bb3 and bb4, then assign the roles to the user.
- Create the u_institution_details table with Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description fields.
- Configure four record ACLs: Read with a script and Branch = EEE, Create with bb2, Write with bb3, and Delete with bb4.
- Verify that bb1 can access only EEE records, a user without roles cannot access records, the administrator can access all records, and a user with bb1+bb2+bb3+bb4 can view, create, edit and delete EEE records.